

ITIS4260 Project Machine Learning 2

April 16, 2025

1 Project Machine Learning 2

ITIS 4260 Efren Antonio

```
[2]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, roc_curve, roc_auc_score, \
    accuracy_score
```

```
[3]: # Read in the data from the CSV file
df = pd.read_csv('datasets/wc1.csv')
pd.set_option('display.max_columns', None)
df.head()
```

```
[3]:
```

	inline_count	external_count	onclick_count	onload_count	onchange_count	\
0	21.0	23.0	1	131.0	0.0	
1	13.0	30.0	2	4.0	1.0	
2	0.0	3.0	1	0.0	0.0	
3	21.0	11.0	1	10.0	1.0	
4	10.0	5.0	1	0.0	0.0	

	avg_inline_script_block	avg_external_script_block	avg onclick_count	\
0	0.0	662.062500	30737	
1	0.0	55.777778	2951	
2	0.0	207.000000	387	
3	0.0	104.800000	1505	
4	0.0	473.000000	4121	

	avg onload_count	avg onchange_count	avg_cyc_complexity	\
0	4	16	0.812500	
1	12	18	0.555556	
2	2	1	1.000000	
3	2	5	0.400000	
4	2	3	1.000000	

	library_code_count	type
0	13	alexa
1	10	alexa
2	1	phish
3	2	alexa
4	3	alexa

```
[4]: # Convert categorical feature into dummy variables with one-hot encoding
df['type_phish'] = (df['type'] == 'phish').astype(int)
df.drop(columns='type', inplace=True)

df.sample(3)
```

	inline_count	external_count	onclick_count	onload_count	\
2304	12.0	11.0	2	6.0	
20905	0.0	0.0	1	0.0	
16834	1.0	7.0	1	0.0	

	onchange_count	avg_inline_script_block	avg_external_script_block	\
2304	0.0	0.0	820.0	
20905	0.0	0.0	0.0	
16834	0.0	0.0	180.0	

	avg_onclick_count	avg_onload_count	avg_onchange_count	\
2304	18447	2	6	
20905	0	0	0	
16834	1870	4	3	

	avg_cyc_complexity	library_code_count	type_phish
2304	1.000000	6	0
20905	0.000000	0	1
16834	0.333333	1	1

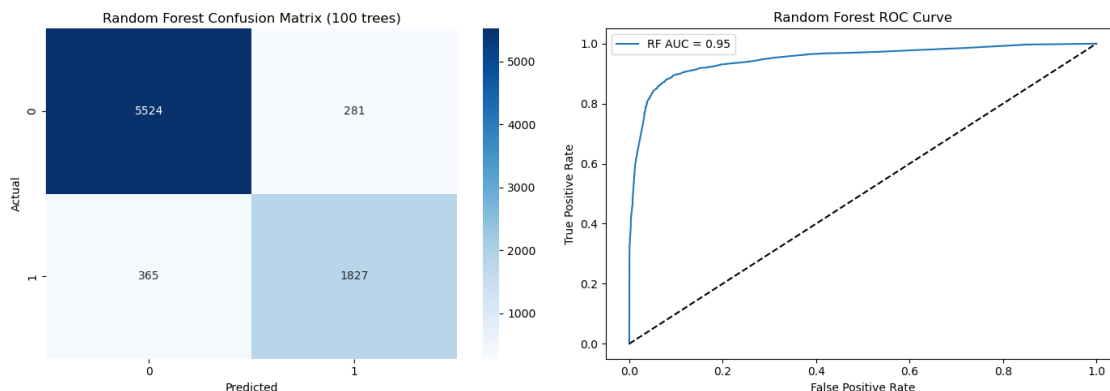
```
[5]: # Split dataset up into train and test sets, random_state is the seed for random
      ↪ number generator
X_train, X_test, y_train, y_test = train_test_split(
    df.drop('type_phish', axis=1), df['type_phish'],
    test_size=0.33, random_state=133)
```

```
[10]: # --- Random Forest with 100 trees ---
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
y_prob_rf = rf.predict_proba(X_test)[: , 1]
```

```
[11]: # Confusion Matrix and ROC for RF
cm_rf = confusion_matrix(y_test, y_pred_rf)
fpr_rf, tpr_rf, _ = roc_curve(y_test, y_prob_rf)
auc_rf = roc_auc_score(y_test, y_prob_rf)
```

```
[12]: # Plot Confusion Matrix and ROC Curve for RF
plt.figure(figsize=(14, 5))
plt.subplot(1, 2, 1)
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Blues')
plt.title("Random Forest Confusion Matrix (100 trees)")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.subplot(1, 2, 2)
plt.plot(fpr_rf, tpr_rf, label=f'RF AUC = {auc_rf:.2f}')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Random Forest ROC Curve")
plt.legend()
plt.tight_layout()
plt.show()
```



```
[13]: # --- Experiment with different number of trees ---
tree_counts = [10, 50, 100, 200, 300]
results = []
for n in tree_counts:
    rf = RandomForestClassifier(n_estimators=n, random_state=42)
    rf.fit(X_train, y_train)
    y_pred = rf.predict(X_test)
    y_prob = rf.predict_proba(X_test)[: , 1]
    acc = accuracy_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_prob)
```

```

        results.append((n, acc, auc))

results_df = pd.DataFrame(results, columns=['n_trees', 'accuracy', 'auc'])
print("\nRandom Forest Performance by Tree Count:")
print(results_df)

```

Random Forest Performance by Tree Count:

	n_trees	accuracy	auc
0	10	0.915468	0.940458
1	50	0.919345	0.947691
2	100	0.919220	0.949288
3	200	0.919470	0.949953
4	300	0.920345	0.950265

```

[14]: # --- Neural Network Classifier ---
nn = MLPClassifier(random_state=1, max_iter=200)
nn.fit(X_train, y_train)
y_pred_nn = nn.predict(X_test)
cm_nn = confusion_matrix(y_test, y_pred_nn)

```

```

[19]: # Try with higher max_iter until convergence
for max_iter in [300, 500, 1000]:
    nn = MLPClassifier(random_state=1, max_iter=max_iter)
    nn.fit(X_train, y_train)
    y_pred_nn = nn.predict(X_test)
    y_prob_nn = nn.predict_proba(X_test)[:, 1]
    cm_nn = confusion_matrix(y_test, y_pred_nn)
    auc_nn = roc_auc_score(y_test, y_prob_nn)
    print(f"\nNeural Network Confusion Matrix (max_iter={max_iter}):")
    print(cm_nn)
    print(f"AUC: {auc_nn:.4f}")
    if nn.n_iter_ < max_iter:
        break

```

Neural Network Confusion Matrix (max_iter=300):

```

[[5313  492]
 [ 705 1487]]
AUC: 0.8587

```

```

[20]: # Plot ROC Curve for final Neural Network
fpr_nn, tpr_nn, _ = roc_curve(y_test, y_prob_nn)
plt.figure(figsize=(6, 5))
plt.plot(fpr_nn, tpr_nn, label=f'NN AUC = {auc_nn:.2f}')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

```

```
plt.title("Neural Network ROC Curve")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

